

Reviewer Pack — Minimal Rank of Geometric Quantization of Functions over Gra...

Entry 4a1497ef1978 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 21:21:00 UTC

Minimal Rank of Geometric Quantization of Functions over Graphs vs Frege Proof Width

Entry ID: 4a1497ef1978 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 21:21:00 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization (over graphs) **Field B** (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

[‘For a graph G with n vertices and m edges, define the geometric quantization of its adjacency matrix as $Q(G)$. The Frege proof width for an unsatisfiable CNF formula F is ω_F . Then, $\omega_F = \Theta(Q(G)^2 \log n)$.’, ‘Equivalently, for any given unsatisfiable CNF formula F with n variables and m clauses, there exists a graph G such that the geometric quantization of its adjacency matrix satisfies $Q(G) \geq c(F) * (\log n)^{-1/2}$, where $c(F)$ is a function of the structure of F .’, ‘Conversely, for any graph G , there exists an unsatisfiable CNF formula F with n variables and m clauses such that $\omega_F \leq d(G)$, where $d(G)$ is a function of $Q(G)$.’]

Rationale (proposer’s reasoning):

[‘Geometric quantization offers a framework to study the properties of functions on geometric spaces. By applying this theory to graph adjacency matrices, we can investigate how the geometric structure of the graph influences the complexity of proving unsatisfiability in CNF formulas.’, ‘Frege proofs are known to be related to the hardness of SAT problems. If our conjecture holds, it would establish a direct connection between the geometric properties of graphs and the complexity of Frege proofs, providing new insights into the nature of computational hardness.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ffd370ab6c5a64af

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given graph G , if the geometric quantization $Q(G)$ of its adjacency matrix satisfies $Q(G)^2 \geq c(F) * (\log n)^{-1/2}$ for a CNF formula F with n variables and m clauses, or if the Frege proof width $\omega_F \leq d(G)$, where $c(F)$ is a function of the structure of F and

$d(G)$ depends on $Q(G)$, then the conjecture is supported. The support condition requires that at least 80% of the seeds produce a correlation coefficient between $Q(G)^2$ and ω_F greater than or equal to 0.8, with an aggregate mean correlat

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - geometric quantization AND adjacency matrix AND graph complexity - Frege proof width AND unsatisfiable CNF formula AND geometric quantization of graphs - quantum graph theory AND Frege proof complexity AND relationship with function minimal rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_cnf(n, m):
    clauses = []
    for _ in range(m):
        clause = [random.randint(1, n) if random.choice([True, False]) else -random.randint(1, n)]
        clauses.append(clause)
    return clauses

def adjacency_matrix(cnf, n):
    adj_matrix = [[0] * n for _ in range(n)]
    for clause in cnf:
        for lit in clause:
            var = abs(lit) - 1
            if lit > 0:
                adj_matrix[var][var] += 1
            else:
                adj_matrix[var][var] -= 1
    return adj_matrix

def geometric_quantization(M):
    n = len(M)
    M2 = [[M[i][j] * M[j][k] for k in range(n)] for j in range(n)]
```

```

trace = sum(M2[i][i] for i in range(n))
det = 1
for i in range(n):
    for j in range(i+1, n):
        sum_col = sum(M2[i][k] + M2[j][k] for k in range(n) if k != i and k != j)
        det *= (M2[i][i] * M2[j][j] - sum_col**2)
return trace / math.sqrt(det)

def frege_proof_width(cnf):
    n = len(cnf)
    m = len(cnf[0])
    # Simplified estimation for demonstration purposes
    return 2 * (n + m) ** 2

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    m = random.randint(n, n**2)
    cnf = generate_cnf(n, m)
    adj_matrix = adjacency_matrix(cnf, n)
    Q_G = geometric_quantization(adj_matrix)
    omega_F = frege_proof_width(cnf)

    metric_value = Q_G ** 2 * math.log(n)
    instances_tested = 1
    conjecture_holds = False
    counterexample = ""

    if metric_value >= omega_F:
        conjecture_holds = True

    return {
        "metric_name": "Q(G)^2 * log n",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")

```

```

elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((seed for seed, result in enumerate(results) if not result["conjecture"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eld0615d.py", line 85, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eld0615d.py", line 60, in run_trial
    Q_G = geometric_quantization(adj_matrix)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eld0615d.py", line 38, in geometric_quantization
    M2 = [[M[i][j] * M[j][k] for k in range(n)] for j in range(n)]
           ^
UnboundLocalError: cannot access local variable 'i' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with proper error handling and debugging to ensure it completes successfully.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13010
2	propose	ollama_remote	glm4:latest	0	0	11564
3	preregistration	ollama_remote	glm4:latest	0	0	6795
4	novelty	ollama_remote	glm4:latest	0	0	4680
5	novelty	ollama_remote	glm4:latest	0	0	5369
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15744
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10176
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11696

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11207
10	judge	ollama_remote	glm4:latest	0	0	15681

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 105923 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/4a1497ef1978.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4a1497ef1978.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4a1497ef1978.tar.gz (if generated)